NAMA: REYHAN GILANG RAMADHAN

KELAS: SI-IIIA

Soal 1 — Tujuan Encapsulation dalam OOP

Jawaban:

Encapsulation adalah salah satu konsep dalam OOP yang berfungsi untuk melindungi data di dalam sebuah class agar tidak bisa diubah secara sembarangan dari luar.

Data tertentu perlu dikendalikan melalui method atau property supaya perubahan data tetap sesuai aturan yang sudah ditentukan. Dengan begitu, data menjadi lebih aman dan kesalahan dalam program dapat dikurangi.

Contohnya, pada sistem rekening mahasiswa, saldo tidak boleh diubah menjadi angka negatif karena dapat menyebabkan kesalahan dalam pencatatan keuangan.

Soal 2 — Private Attribute dan Property

Jawaban:

Attribute dengan awalan __ digunakan untuk membuat attribute yang bersifat private. Tujuannya agar data tidak mudah diakses atau diubah secara langsung dari luar class.

Sedangkan `@property` berfungsi untuk membaca atau mengambil nilai dari private attribute dengan cara yang lebih aman.

`@<nama>.setter` digunakan untuk mengatur atau mengubah nilai attribute dengan menambahkan aturan validasi sebelum data disimpan.

Jadi, ketiganya membantu menjaga keamanan data dan mengontrol perubahan nilai di dalam sebuah class.

Soal 3 — Prediksi Output

Jawaban:

Output yang muncul ketika program dijalankan adalah:

```
3
```

Penjelasan:

Output tersebut muncul karena object `mhs` dibuat dengan nama Jhon Doe dan nilai SKS sebesar 3.

Attribute `__sks` merupakan private attribute yang digunakan untuk menyimpan jumlah SKS mahasiswa.

Property `@property` dengan nama `sks` berfungsi untuk mengambil nilai dari `__sks` melalui method:

```
@propertydef sks(self):    return self.__sks
```

Ketika perintah `print(mhs.sks)` dijalankan, Python akan memanggil property `sks` untuk membaca nilai private attribute tersebut.

Dengan demikian, nilai 3 dapat ditampilkan tanpa harus mengakses private attribute secara langsung.

Soal 4 — Validasi Setter

Jawaban:

Berikut kode yang sudah dilengkapi:

```
class Rekening:    def __init__(self, saldo):        self.__saldo = 0
self.saldo = saldo    @property    def saldo(self):        return
self.__saldo    @saldo.setter    def saldo(self, nilai):        if nilai <
0:            raise ValueError("Saldo tidak boleh negatif")
self.__saldo = nilairekening = Rekening(500000)print(rekening.saldo)
```

Output:

```
500000
```

Jika saldo diisi dengan nilai negatif, misalnya:

```
rekening.saldo = -100000
```

Maka akan muncul error:

```
ValueError: Saldo tidak boleh negatif
```

Penjelasan:

- `@property` digunakan untuk mengambil nilai saldo.
- `@saldo.setter` digunakan untuk mengubah nilai saldo.
- `if nilai < 0` digunakan untuk memeriksa apakah saldo bernilai negatif.

- `raise ValueError` digunakan untuk menampilkan error jika saldo tidak sesuai aturan.

Dengan adanya validasi tersebut, saldo tidak dapat diubah menjadi angka negatif.

Soal 5 — Analisis Access Control

Jawaban:

Jika attribute `saldo` dapat diubah secara langsung tanpa validasi, maka data rekening berisiko mengalami kesalahan. Misalnya, seseorang dapat mengubah saldo menjadi angka negatif atau memasukkan nilai yang tidak sesuai dengan aturan sistem.

Hal tersebut dapat menyebabkan pencatatan keuangan menjadi tidak akurat dan merugikan pemilik rekening.

Contoh aturan validasi untuk rekening mahasiswa:

Saldo rekening mahasiswa tidak boleh bernilai negatif. Jika mahasiswa ingin melakukan penarikan uang, jumlah penarikan tidak boleh melebihi saldo yang tersedia.

Contohnya, jika saldo mahasiswa sebesar Rp500.000, maka mahasiswa hanya boleh menarik uang maksimal Rp500.000.

Dengan aturan tersebut, sistem dapat menjaga saldo agar tetap aman dan menghindari kesalahan dalam transaksi.